



Programação Avançada 2025/26

Técnicas de Refactoring (1/2) : *Extract Method, Extract Class, Replace Inheritance with Delegation e Replace Magic Number with Symbolic Constant*

Bruno Silva, Patrícia Macedo

Sumário



- Classificação das Técnicas de Refactoring, segundo *Martin Fowler*;
- Qual a técnica a aplicar?
- Exemplos desta aula (★) + atividades:
 - Extract Method;
 - Extract Class;
 - Replace Inheritance with Delegation;
 - Replace Magic Number with Symbolic Constant;

Classificação das Técnicas de Refactoring

⚠ As técnicas de refactoring servem para resolver problemas identificados (code smells). Cada code smell é tratado por uma ou mais técnicas de refactoring.

Martin Fowler categorizou as técnicas refactoring em **6 categorias**:

- Composing Methods;
- Moving Features Between Objects;
- Organizing Data;
- Simplifying Conditional;
- Making Method Calls Simpler;
- Dealing with Generalization.

Composing Methods

Estas técnicas de refactoring têm como objectivo resolver problemas relacionados com métodos demasiado extensos/curtos e código duplicado. Tal como o nome sugere os métodos são decompostos e compostos para melhorar o desenho da aplicação.

- Extract Method ★
- Inline Method
- Replace Temp with Query
- Split Temporary Variables
- Replace Method with Method Object
- Remove Assignment to Parameters
- Substitute Algorithm

Moving Features Between Objects

Estas técnicas de refactoring tem como objectivos: (i) a transferência de funcionalidade entre classes; (ii) a criação de novas classes, e; (iii) o encapsulamento de detalhes de implementação.

- Move Method
- Move Field
- Extract Class ★
- Inline Class
- Hide Delegate
- Remove Middle Man
- Introduce Foreign Method
- Introduce Local Extension

Organizing Data

Estas técnicas de refactoring têm como objetivo tornar a manipulação dos dados mais eficiente.

- Self Encapsulate Field
- Replace Data Value with Object
- Change Value to Reference
- Change Reference to Value
- Replace Array with Object
- Duplicate Observed Data
- Change Unidirectional Association to Bidirectional
- Change Bidirectional Association to Unidirectional
- Replace Magic Number with Symbolic Constant ★

Simplifying Conditional Expressions



Estas técnicas de refactoring têm como objectivo melhorar a complexidade introduzida pelas expressões condicionais.

- Decompose Conditional
- Consolidate Conditional Expression
- Consolidate Duplicate Conditional Fragments
- Remove Control Flag
- Replace Nested Conditional with Guard Clauses (! ver fim slides)
- Replace Conditional with Polymorphism
- Introduce Null Object
- Introduce Assertion

Making Method Calls Simpler

Estas técnicas de refactoring têm como objectivo tornar as chamadas aos métodos mais simples e fáceis de compreender.

- Rename Method
- Add Parameter
- Remove Parameter
- Separate Query from Modifier
- Preserve Whole Object
- Introduce Parameter Object
- Hide Method
- Replace Constructor with Factory Method (🤔 lembra algo?)
- Replace Error Code with Exception
- Replace Exception with Test

Dealing with Generalization

Estas técnicas de refactoring têm como objetivo mover funcionalidades ao longo da hierarquia de classes, criando novas classes e interfaces e substituindo herança por delegação e vice-versa.

- Pull Up Field
- Push Down Method
- Push Down Field
- Extract Subclass
- Extract Superclass
- Collapse Hierarchy
- Form Template Method
- Replace Inheritance with Delegation ★
- Replace Delegation with Inheritance

Qual a técnica a aplicar ?

e.g., <https://refactoring.guru/refactoring/smells>

Para cada **code smell** existe a indicação de um conjunto de técnicas de refactoring associadas.

Dentro das técnicas propostas seleciona-se a mais adequada à situação concreta.

[/ Refactoring / Code Smells / Bloaters](#)

Data Clumps

Signs and Symptoms

Sometimes different parts of the code contain identical groups of variables (such as parameters for connecting to a database). These clumps should be turned into their own classes.

Treatment

- If repeating data comprises the fields of a class, use **Extract Class** to move the fields to their own class.
- If the same data clumps are passed in the parameters of methods, use **Introduce Parameter Object** to set them off as a class.
- If some of the data is passed to other methods, think about passing the entire data object to the method instead of just individual fields. **Preserve Whole Object** will help with this.
- Look at the code used by these fields. It may be a good idea to move this code to a data class.

Extract Method

Aplica-se para resolver vários tipos de problemas: código duplicado, métodos longos, switch statements, etc...

É também usado como um passo intermédio em técnicas mais complexas (Form Template Method, Hide Method etc..)

- **Sumário:** Um método tem um fragmento de código que pode ser agrupado, então um novo método é criado com esta porção de código e referenciado no método original.
- **Mecanismo:**
 - i. Criar um novo método e dar-lhe o nome da intenção do método.
 - ii. Copiar o código extraído do método da fonte para o novo método de destino.
 - iii. Verificar as referências às variáveis.
 - iv. Substituir o código extraído no método de origem por uma chamada ao método de destino.

Extract Method | Exemplo

```
void printOwing() {  
    printBanner();  
  
    // Print details.  
    System.out.println("name: " + name);  
    System.out.println("amount: " + getOutstanding());  
}
```



```
void printOwing() {  
    printBanner();  
    printDetails(getOutstanding());  
}  
  
void printDetails(double outstanding) {  
    System.out.println("name: " + name);  
    System.out.println("amount: " + outstanding);  
}
```

Atividade 1

Repositório:

https://github.com/estsetubal-pa/ExtractMethod_Template

 **Code Smell:** *Duplicated Code*

 **Técnica de refactoring:** *Extract Method*

Extract Class

- **Sumário:** Tem-se uma classe faz o trabalho de duas (ou mais), então cria-se uma nova classe e coloca-se nela os atributos e métodos responsáveis pela funcionalidade relevante.
- **Mecanismo:**
 - i. Decidir sobre a forma exata como pretende dividir as responsabilidades da classe.
 - ii. Criar uma nova classe para conter a responsabilidade relevante.
 - iii. Criar uma relação entre a classe antiga e a nova.
 - iv. Use **Move Field** e **Move Method** para cada atributo e método que tenha decidido mudar para a nova classe.
 - v. Pensar também na acessibilidade ao novo par de classes a partir do exterior. Pode esconder a classe do cliente por completo, tornando-a privada ou pode torná-la pública, permitindo ao cliente alterar directamente os valores.

Extract Class | Exemplo

```
public class Person {  
    private String name, streetAddress, cityName;  
    public String getName() { return this.name; }  
    public String getCityName() { return this.cityName; }  
    public String getStreetAddress() { return this.streetAddress; }  
}
```

⬇ Que *code smell* identificaria acima que levou a este refactoring?

```
public class Person {  
    private String name;  
    private Address address;  
    public String getName() { return this.name; }  
    public String getCityName() { return this.address.getCityName(); }  
    public String getStreetAddress() { return this.address.getStreetAddress(); }  
}  
public class Address {  
    private String streetAddress, cityName;  
    public String getCityName() { return this.cityName; }  
    public String getStreetAddress() { return this.streetAddress; }  
}
```

Atividade 2

Repositório:

https://github.com/estsetubal-pa/ExtractClass_Template

 **Code Smell:** *Large Class*

 **Técnica de refactoring:** *Extract Class* - de forma a fazer uma correta separação das responsabilidades.

No fim deverá ter 3 classes:

- `AppMainCalculator` - classe responsável por lançar a aplicação;
- `CalculatorUI` - classe responsável por implementar a interação com o utilizador;
- `Calculator` - classe responsável por implementar a lógica da calculadora

Replace Inheritance with Delegation

 Aplica-se ao *bad smell* refused bequest.

- **Sumário:** uma subclasse B utiliza apenas um subconjunto de atributos e/ou métodos da superclasse A.
- **Mecanismo:**
 - i. substituir a herança por *composição*, adicionando um atributo do tipo A à classe B;
 - ii. Criar os métodos respetivos desse comportamento e delegar para os métodos de A.

 Altera uma relação "is-a" para uma relação "has-a", coerente com o que foi potencialmente idealizado;

 Se a herança for desejável, rever a hierarquia estabelecida. A superclasse não conterá mais do que deveria?

Replace Magic Number with Symbolic Constant

Sumário: O código utiliza um número que tem um significado associado.

Mecanismo: substituir esse número por uma constante cujo nome é explanatório.

```
double potentialEnergy(double mass, double height) {  
    return mass * height * 9.81;  
}
```



```
static final double GRAVITATIONAL_CONSTANT = 9.81;  
double potentialEnergy(double mass, double height) {  
    return mass * height * GRAVITATIONAL_CONSTANT;  
}
```

👍 É mais fácil substituir o valor de uma constante do que todas as ocorrências desse valor no código.

Atividade 3

Repositório:

https://github.com/estsetubal-pa/JavaRefactoring_ReplaceInheritanceDelegation



Code Smells: *Refused Bequest + Magic Number*



Técnicas de refactoring: *+ Replace Inheritance with Delegation + Replace Magic Number with Symbolic Constant*

Consulta Autônoma

! Replace Nested Conditional with Guard Clauses:

- <https://www.youtube.com/watch?v=CFRhGnuXG-4>
-

Referências de Catálogos (Code smells e técnicas):

- <https://refactoring.guru/smells>
- <https://refactoring.guru/refactoring/techniques>